

CraveDrop Security Assessment and Secure OIDC Login

Hansaja A. M. G (IT22171856)

Dharmasiri I. D. N. D (IT22254320)

Sanjeewa P. D. L. B (IT22629708)

Aluthwaththa A. W. D. S. M (IT22267290)

September 2026

1 Executive summary

This report documents a security assessment of CraveDrop, a food-ordering and delivery microservice application. The team preserved the vulnerable application at immutable tag `baseline-vulnerable`. The tag resolves to upstream commit `cb68a377f5b8cdc3b12f86883ac6fb703ef5e405`.

The assessment reproduced, fixed, and regression-tested seven distinct weaknesses. The work also adds one WSO2 Identity Server customer-login feature. The feature uses OpenID Connect Authorization Code flow with S256 PKCE and a CraveDrop HttpOnly cookie session. It is a feature, not an eighth counted vulnerability.

All tests use synthetic fixtures. Evidence files contain redacted request and response data only. The detailed evidence index is `docs/finding-matrix.md`.

2 Project eligibility and method

The original project is [nmdra/CraveDrop](#). It is an MIT-licensed food-ordering and delivery application. The selected default-branch baseline is dated 17 July 2025. The assessed work predates the July 2026 semester start.

The method used a fixed baseline, deterministic Customer A and Customer B fixtures, live baseline requests, focused code changes, and negative plus positive regression controls. A finding was counted only when it had a distinct root cause, a baseline attack, an impact, a focused fix, a blocked attack, an authorized control, and a residual-risk statement.

3 System and trust boundaries

CraveDrop uses separate user, order, payment, delivery, and driver services. The API gateway routes browser requests to these services. The services use PostgreSQL or MongoDB, depending on the service. The security fixes enforce identity and authorization at the affected service boundary. The gateway is not treated as the only authorization control.

4 Finding summary

ID	Weakness	Mapping	Result
V1	Missing payment authentication	CWE-306, OWASP API1	Anonymous payment actions return 401.
V2	Order IDOR or BOLA	CWE-639, OWASP API1	Foreign customer access is blocked.
V3	Order mass assignment	CWE-915, OWASP API3	Server-owned fields are rejected.
V4	Delivery mutation authorization	CWE-862, OWASP API5	Only the assigned driver can mutate delivery state.
V5	Driver administration authorization	CWE-862, OWASP API5	A driver can update only own availability.
V6	Payment amount and state manipulation	CWE-841, API business logic	The server calculates price and owns payment state.
V7	Token exposure and session design	CWE-922, CWE-532, OWASP API2	Tokens leave JSON, storage, and logs.

4.1 V1: Missing authentication on payment actions

Baseline attack. An anonymous request to `POST /api/payments/create-payment-intent` reached the payment handler. It did not receive an authentication error.

Impact. An attacker can invoke a payment action without a customer identity. This can consume payment-provider resources and bypass the intended customer boundary.

Fix and result. Payment routes now require a verified Bearer JWT. The caller identity comes from the verified token. The anonymous request returns 401. An authenticated synthetic customer reaches the payment-start handler.

Residual risk. The local control reaches a synthetic Stripe error. It does not prove a real card charge.

4.2 V2: Order IDOR or BOLA

Baseline attack. An unauthenticated request read Customer A's synthetic order by order ID.

Impact. An attacker can view or alter another customer's order if the order ID is known.

Fix and result. Order reads and mutations require an authenticated customer. The query includes both order ID and verified customer ID. Customer B receives 404 for Customer A's order. Customer A can read the owned order.

Residual risk. Every new order route must apply the same ownership scope.

4.3 V3: Order mass assignment

Baseline attack. An unauthenticated order update supplied `userId`, `totalAmount`, `status`, and `createdAt`. The service persisted client-controlled values.

Impact. A caller can alter ownership, price, lifecycle state, or audit data.

Fix and result. The controller rejects protected fields and allow-lists customer-editable values. The server owns identity, price, payment state, and timestamps. Protected fields return 400. Valid authorized updates and order creation succeed.

Residual risk. New editable fields require explicit review before they join the allow-list.

4.4 V4: Unauthorized delivery mutation

Baseline attack. An anonymous caller changed a delivery status. The same boundary accepted a location update.

Impact. An attacker can falsify delivery progress or tracking data.

Fix and result. Delivery writes require a valid driver JWT, an assigned-driver match, and an allow-listed state transition. Anonymous, customer, and unrelated-driver writes fail. The assigned driver can make the valid status and location updates. A reverse transition returns 409.

Residual risk. The fix does not prove the integrity or accuracy of the driver device and GPS source.

4.5 V5: Broken administrative authorization

Baseline attack. An unauthenticated request changed Driver B's availability through Driver B's ID.

Impact. An attacker can disrupt delivery allocation.

Fix and result. Driver availability updates require a verified driver JWT. The verified driver ID must match the URL ID. A driver cannot update another driver's availability. A driver can update own availability.

Residual risk. This finding covers driver availability. Restaurant administration remains outside this counted fix.

4.6 V6: Payment amount and state manipulation

Baseline attack. A client created a card order with amount 1 instead of the fixture total 2500. The order was also marked paid.

Impact. A customer can lower the charge or obtain a paid order without confirmed payment.

Fix and result. The order service calculates the USD price from trusted product data. It ignores client currency and paid-status fields. New card orders start pending. A tampered request persists 2500 USD and pending status.

Residual risk. Payment confirmation must remain bound to a trusted order and verified provider event.

4.7 V7: Sensitive token exposure and session design

Baseline attack. Login and refresh responses returned bearer tokens in JSON. The service logged refresh-token data. The frontend stored access tokens.

Impact. Script injection, browser access, or log access can expose bearer tokens and allow session impersonation.

Fix and result. Login and refresh issue strict HttpOnly cookies. The frontend does not store bearer tokens. Logging omits request query values, which also prevents OIDC authorization-code logging. Login and refresh JSON omit access tokens. Cookie mutations require the approved Origin.

Residual risk. XSS can still make same-origin requests while a user session exists. Future work includes Content Security Policy and broader XSS review.

5 WSO2 OpenID Connect customer login

The added feature supports one customer login with WSO2 Identity Server 7.1.0. CraveDrop generates state, nonce, and a PKCE verifier. It uses an S256 challenge and stores the temporary flow in a signed short-lived HttpOnly cookie. The callback exchanges the authorization code server-side and validates issuer, audience, RS256 signature, expiry, nonce, and subject.

The local account key is provider issuer plus subject. The service does not link a local account by matching email address. If the local provider does not release a verified email claim, the service stores a deterministic non-deliverable identity placeholder. The real synthetic-user control returned a safe dashboard redirect, issued strict HttpOnly CraveDrop cookies, cleared the flow cookie, and authorized a protected endpoint. Node 20 mocked controls reject tampered state before code exchange and reject invalid nonce or ID-token input.

6 Supporting tools and secure development

The team ran `npm audit` for user, order, payment, delivery, and driver services. The results are recorded in `evidence/tools/npm-audit.txt`. These dependency advisories are not counted vulnerabilities because the team did not reproduce an endpoint impact.

The OWASP ZAP local-gateway scan is deferred because its Docker image download and scan exceed the available bandwidth. No ZAP warning or scan result is claimed. The team must run a bounded scan before submission if bandwidth is available.

Preventive practices include dependency review, server-owned sensitive state, explicit field allow-lists, endpoint-level authorization, redacted evidence, synthetic data, focused negative and positive tests, and protected browser sessions.

7 Evidence and contributions

The source record for each finding is `docs/report-notes.md`. The evidence matrix links the baseline request, after-fix proof, test, and commit. The main fix commits are 4904430, 2f58bbf, and 8505d31. The OIDC feature commit is 4a440d1.

Hansaja implemented the user-session hardening and WSO2 login. Dharmasiri implemented order authentication, BOLA, and mass-assignment controls. Sanjeewa implemented delivery, driver, and payment-integrity controls. Aluthwaththa prepared fixtures, tool evidence, report material, and packaging documentation. The final video and Git history must show the actual contributions.

8 Conclusion

The assessment preserves a reproducible vulnerable baseline and demonstrates seven distinct fixed vulnerabilities. The application now enforces authentication, ownership, server-owned payment state, delivery authorization, and cookie-session handling at the affected boundaries. A secure WSO2 customer-login path is available for local demonstration. Remaining risks and the deferred ZAP scan are stated without unsupported claims.